#Architectural Ledger

Vendor Management System Portal

Java Application · RAG Pipeline · Qwen:0.5B (Ollama) · Pinecone · nomic-embed-text

PURE JAVA APPLICATION

QWEN:0.5B OLLAMA LLM

NOMIC-EMBED-TEXT

PINECONE VECTOR DB

CONTRACT COMPARE

POSTGRESQL DATABASE

Team: Atharva Abhijeet Mahalkar| Japkeerat Kaur | Prakhar Vishwakarma · Guide: Prof. Suhas Joshi · April 2026

Catalog

Architectural Ledger: Vendor Management System Portal

01 1. Problem Statement

02 2. System Architecture (100% Java)

03 3. RAG Pipeline Implementation

04 4. Tools & Technologies

05 5. Project Modules

06 6. Application Screenshots

07 7. Use Cases

08 8. Challenges & Solutions

09 9. Future Scope

10 10. Conclusion

Problem Statement

Identifying Core Business and Technical Challenges

01 Business Problem

- Vendor contracts scattered — no central searchable store
- Extracting specific clauses requires reading full documents
- No automated contract comparison for risk assessment
- Compliance checks done manually — slow and error-prone

02 Technical Problem

- Keyword search fails on semantic meaning
- No vector DB for similarity-based document retrieval
- No LLM to generate grounded answers from docs
- Cloud AI APIs = privacy risk + billing cost

Beyond Keyword Search: From Finding to Understanding

The Old Way

The "Ctrl + F" Approach

- **Exact Match Only:** Finds a word, but not the meaning.
- **Manual Sifting:** You find 20 instances of "Payment," but you still have to read all 20 to find the actual terms.
- **Fragmented View:** Can only search one document at a time.
- **Zero Context:** It tells you where a word is, but not what it means in the bigger picture.

The Result: **Information Retrieval**

The New Way

Our AI-Powered VMS

- **Semantic Understanding:** Finds "Payment Terms" even if the document says "Remittance Schedule."
- **Instant Answers:** Doesn't give you a list of pages; it gives you a concise, accurate answer.
- **Cross-Document Intelligence:** Analyzes contracts, invoices, and logs simultaneously.
- **Contextual Synthesis:** Compares two different vendors or documents automatically.

The Result: **Knowledge Extraction**

> *Ctrl + F finds the needle in the haystack. Our project explains what the needle is doing.*

System Architecture 100% Java

Multi-layered Architecture for Secure and Scalable Vendor Management

FRONTEND (HTML / CSS / JS)

Static files served by Java HTTP Server — all 6 portal pages

↓

JAVA APPLICATION LAYER

Routing, Vendor CRUD (PostgreSQL/JDBC), AI orchestration

↓

JAVA AI SERVICES

IngestService · SearchService · CompareService (HTTP to Ollama & Pinecone)

↓

OLLAMA (localhost:11434)

Serves Qwen:0.5B (generate) + nomic-embed-text (embeddings) locally

↓

PINECONE + POSTGRESQL

Pinecone: 768-dim vector index with namespaces | PostgreSQL: vendor data

RAG Pipeline Implemented in Java

End-to-End Document Intelligence via Local LLM and Vector Search

INGESTION (Java IngestService):

Upload Doc → Java chunks text (500 tok) → HTTP POST to Ollama /embeddings → HTTP POST to Pinecone /upsert

QUERY (Java SearchService):

User Query → HTTP POST Ollama /embeddings → HTTP POST Pinecone /query → Build prompt in Java → HTTP POST Ollama /generate

Note: All AI calls made from Java using **HttpURLConnection** — no Python, no Flask, no LangChain required. **OllamaClient.java** and **PineconeClient.java** handle all HTTP communication.

Tools and Technologies

Key Platform Changes & Strategic Implications for 2025

Java (JDK 11+)
Entire application — backend, RAG, HTTP, JDBC

Qwen:0.5B (Ollama)
Local LLM — answer generation via HTTP from Java

nomic-embed-text
768-dim embeddings called from Java via Ollama REST

Pinecone
Cloud vector DB — Java calls REST API for upsert/query

PostgreSQL 15
Relational DB — vendor & user data via JDBC

HTML / CSS / JS
Frontend portal — served statically by Java server

HttpURLConnection
Java built-in HTTP client for Ollama & Pinecone calls

org.json / Json
JSON parsing for API responses in Java

Project Modules

Functional Breakdown of the Architectural Ledger System

01 Central Ledger Intelligence

Java SearchService: Ollama embed → Pinecone → Qwen:0.5B answer

02 Document Repository

Java IngestService: chunk → Ollama embed → Pinecone upsert

03 Document Comparison

Java CompareService: dual Pinecone retrieval → Qwen clause diff

04 Vendor Management

Java Vendor: full CRUD via PostgreSQL JDBC

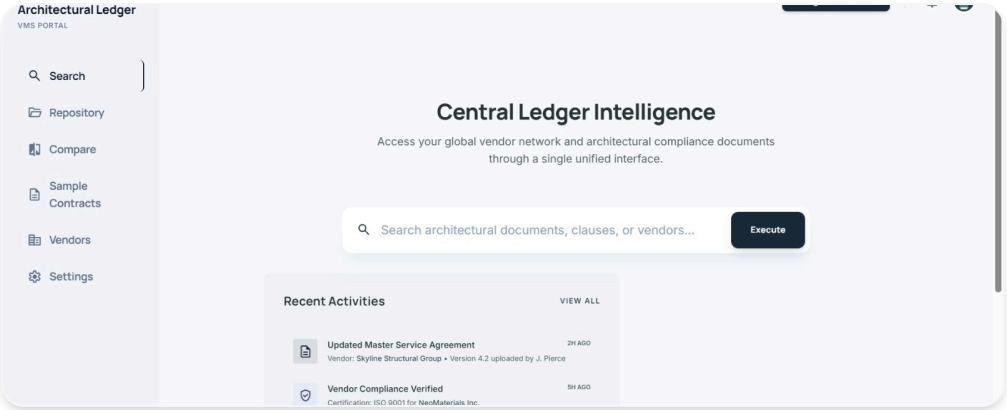
05 Sample Contracts

Reference library for AI comparison benchmarks

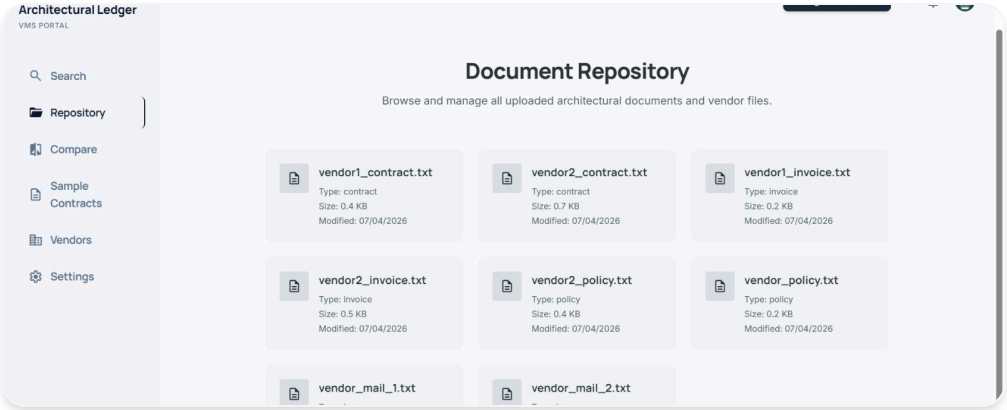
06 Settings

config.properties: Ollama model, Pinecone key, chunk settings

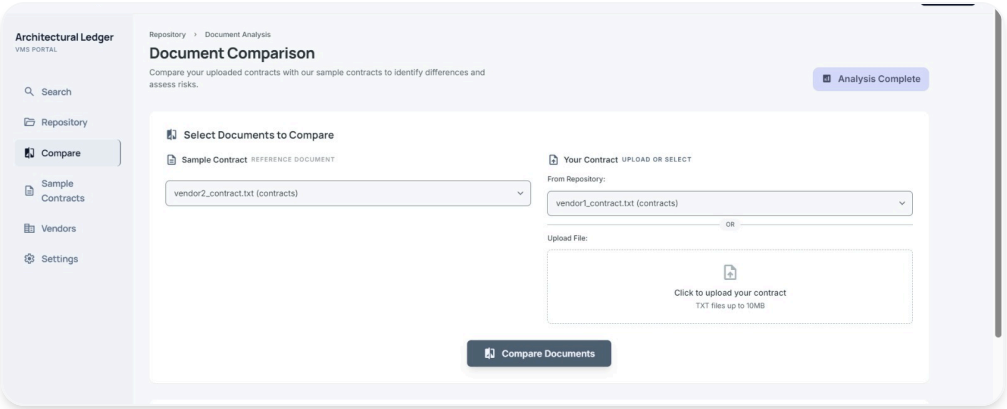
Application Screenshots



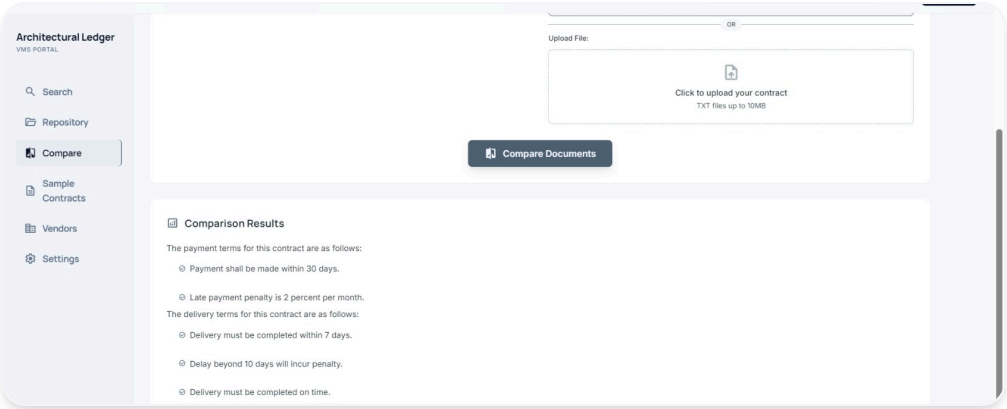
1. AI Search — Java SearchService calls Ollama + Pinecone



2. Repository — Java IngestService indexes docs in Pinecone



3. Document Comparison — Java CompareService selects contracts



4. Comparison Results — Qwen:0.5B extracts clauses via Java

Use Cases

Real-world Scenarios and Workflow Implementations

UC1 — AI Contract Query

Actor: Procurement Manager

Flow: Query → Java embeds (Ollama) → Pinecone → Qwen:0.5B answers

Result: ✓ Clause answer in 2–5 seconds

UC2 — Contract Risk Compare

Actor: Legal Team

Flow: Select 2 docs → Java retrieves from Pinecone → Qwen extracts

Result: ✓ Payment, delivery & penalty clauses

UC3 — Compliance Audit

Actor: Compliance Officer

Flow: Search policy term → Pinecone retrieves all policy docs → Qwen

Result: ✓ Consolidated compliance summary

Challenges and Solutions

Overcoming Technical Hurdles in Java-Native RAG Implementation

Qwen:0.5B Context Limit	→	Reduced chunk size to 400 tokens; top-K=3 for comparison tasks
Parsing Ollama Streamed JSON	→	Read full HTTP response stream in Java; used org.json to parse result
Pinecone Namespace Isolation	→	Passed user namespace in every Java upsert & query call in PineconeClient
Pinecone Free Tier Batch Limit	→	Batched upserts in groups of 100 vectors in Java IngestService

Future Scope

Strategic Roadmap for System Evolution and Enhanced Capabilities

User Auth + Namespaces

JWT login in Java — per-user Pinecone namespace, isolated search

PDF & DOCX Parsing

Apache PDFBox + Apache POI for Java-native document parsing

Automated Risk Scoring

Qwen generates 0–100 risk score per contract in Java

Multi-model Support

Switch Qwen model via config.properties — no code change

Docker / Cloud Deploy

Docker container on AWS/GCP — GPU Ollama + Pinecone prod tier

Fine-tuned Qwen

Fine-tune on legal/procurement text for higher clause accuracy

Conclusion

Architectural Ledger is a 100% Java application — the RAG pipeline calls Ollama and Pinecone directly via HTTP, with no Python or external scripting layer required.

Key Achievements:

- **Pure Java RAG** — OllamaClient.java & PineconeClient.java handle all AI HTTP calls
- **Qwen:0.5B via Ollama** — local LLM, CPU-only, ~1.2GB RAM, 2–5s response time
- **Pinecone** — 768-dim cosine vector DB, per-user namespaces, sub-100ms retrieval
- **PostgreSQL 15** — vendor & document data via JDBC; full CRUD with referential integrity

Thank You · Questions Welcome